ISSUE 32
Design Shared Simulation Core #32

June 16, 2026

1 Objective

The purpose of this document is to define the common execution core used for candle-by-candle strategy simulation.

The execution core must provide a reusable simulation engine that can be shared between:

- Simulation Engine (Backtesting)
- Trading Bot (Validation on unseen data)

The execution logic must be completely separated from:

- Data source implementation
- Strategy implementation details
- Metrics calculation
- Database persistence
- Output formatting

Its sole responsibility is to execute trading strategies on a sequence of market candles and maintain the virtual portfolio state.

2 Core Simulation Flow

2.1 Overview

The execution process follows a deterministic candle-by-candle simulation model.

```
Load Input Data
|
v
Initialize Portfolio State
|
v
For Each Candle
|
v
Build Execution Context
|
```

```
v
Call func. "Strategy .on\_candle()"
|
v
Receive Signal
|
v
Execute Signal
|
v
Update Portfolio
|
v
Update Trade Log
|
v
Next Candle
|
v
Close Remaining Positions
|
v
Generate Execution Result
```

2.2 Detailed Execution Steps

Step 1 – Initialization

The Execution Engine receives:

```
strategy
candles
initial_capital
```

The following objects are initialized:

```
PortfolioState
TradeLog
ExecutionContext
```

Step 2 – Candle Iteration

The engine processes candles sequentially:

```
for candle in candles:
```

Step 3 – Context Creation

For every candle, an ExecutionContext instance is created containing:

- Current candle
- Historical candles
- Current portfolio state

NOTE:

Context is an object that contains all the information available to the strategy at the current moment in time. In more detail, for example some strategies require several past candles (that's why it's called context for current candle).

ExecutionContext is a concrete Context implementation for our Execution Core.

Step 4 – Strategy Execution

The engine invokes the strategy:

```
signal = strategy.on_candle(context)
```

NOTE:

Basically we get the signal that strategy provides on some exact candle considering the "context" of the current state.

Step 5 – Signal Processing

The generated signal is passed to the Order Executor.

Supported signals (for now):

- BUY
- SELL
- HOLD

Step 6 – Portfolio Update

The execution core updates:

- Available cash
- Open positions
- Equity
- Realized P&L
- Unrealized P&L

NOTE:

So basically here is what happens after a strategy signal is executed.

Let's say the strategy says:

```
return BuySignal()
```

The Execution Core has received this signal and should simulate a buy.

After this, the virtual account state (Portfolio State) changes.

Step 7 – Trade Logging

Whenever a position is closed, a trade record is added to the Trade Log.

Step 8 – Finalization

After the last candle is processed:

```
close_open_positions()
```

may be executed depending on simulation rules.

Step 9 – Result Generation

The execution engine returns an ExecutionResult object.

3 Shared Execution Model for on_candle()

3.1 Purpose

All trading strategies must communicate with the execution core through a single unified interface.

The execution engine must not depend on any specific strategy implementation such as:

- SMA
- EMA
- RSI
- MACD
- Sentiment Analysis
- Machine Learning Models

The only required interaction point is:

```
on_candle()
```

3.2 Strategy Interface

```
class Strategy(ABC):


@abstractmethod
def on_candle(
    self,
    context: ExecutionContext
) -> Signal:
    pass
```

3.3 Execution Context

Strategies receive an ExecutionContext object.

```
class ExecutionContext:


current_candle
```

```
historical_candles

portfolio_state
```

Example usage:

```
context.current_candle.close

context.historical_candles[-20:]

context.portfolio_state.cash
```

3.4 Returned Signals

A strategy must return a Signal object.

Supported signal types:

```
BuySignal

SellSignal

HoldSignal
```

Example:

```
def on_candle(self, context):


if buy_condition:
    return BuySignal()

if sell_condition:
    return SellSignal()

return HoldSignal()
```

3.5 Benefits

This design follows the Open/Closed Principle.

New strategies can be added without modifying the execution engine.

4 Portfolio State Structure

4.1 Purpose

The execution core maintains all account information through a unified PortfolioState object.

4.2 PortfolioState Definition

```
@dataclass
class PortfolioState:


cash: float

position_size: float
```

```
entry_price: float

realized_pnl: float

unrealized_pnl: float

equity: float
```

4.3 Field Description

cash Available capital not currently invested.

position_size Size of the currently open position.

entry_price Entry price of the active position.

realized_pnl Profit or loss from closed trades.

unrealized_pnl Profit or loss from currently open positions.

equity Total account value.
[equity = cash + market_value_of_position]

4.4 Position Model

```
@dataclass
class Position:


symbol: str

quantity: float

entry_price: float

opened_at: datetime
```

5 Input and Output Contracts

5.1 Execution Input Contract

The execution engine receives an ExecutionRequest object.

```
@dataclass
class ExecutionRequest:


strategy: Strategy

candles: List[Candle]

initial_capital: float
```

5.2 Candle Contract

```
@dataclass
class Candle:


timestamp: datetime

open: float

high: float

low: float

close: float

volume: float
```

5.3 Execution Output Contract

The execution engine returns an ExecutionResult object.

```
@dataclass
class ExecutionResult:


portfolio_state: PortfolioState

trade_log: List[Trade]
```

5.4 Trade Contract

```
@dataclass
class Trade:


entry_time: datetime

exit_time: datetime

entry_price: float

exit_price: float

quantity: float

pnl: float
```

5.5 Responsibility Boundary

The execution engine is NOT responsible for calculating:

- Sharpe Ratio
- Win Rate

- Maximum Drawdown
- Top-N Ranking

These calculations belong to the Analytics Module.
The execution engine only returns:

- Trade Log
- Portfolio State

6 Documentation Update

The project documentation must include the following sections:

1. Execution Core Overview
2. Simulation Flow
3. Strategy Contract
4. Portfolio State
5. Input/Output Contracts
6. Reuse Between Modules

6.1 Execution Core Overview

```
Strategy
|
v
Execution Engine
|
v
Trade Log
|
v
Analytics Module
```

6.2 Reuse Between Modules

```
Simulation Engine
|
v
Execution Core
^
|
Trading Bot
```

6.3 Architectural Rule

Simulation Engine and Trading Bot must never implement their own candle-processing loop.

All candle-by-candle execution logic must be delegated to the shared Execution Core.
This guarantees:

- Code reuse
- Consistent execution behavior
- Deterministic simulation results
- Easier maintenance
- Simplified testing